

Constraints

- Key Constraint (primary key vs unique)
- Referential Integrity Constraint (FK)

$$R \subseteq S \Rightarrow R - S = \emptyset$$

$$R = \emptyset \Rightarrow R \subseteq \emptyset$$

SQL Commands

Insert

```
INSERT INTO
  StarsIn (movieTitle, movieYear)
VALUES
  ("Falcon", 1942);
```

```
INSERT INTO
  Studio (name)
SELECT DISTINCT
  studioName
FROM
  Movies
WHERE
  studioName NOT IN (
    SELECT
      name
    FROM
      Studio
  );
```

Delete

```
DELETE FROM StarsIn
WHERE
  movieTitle = "The Flacon";
```

Update

```
UPDATE MovieExec
SET
  name = "Pres. " || name
WHERE
  cert IN (
    SELECT
      presC
    FROM
      Studio
  );
```

|| is concat

Foreign Keys

```
CREATE TABLE Studio (
  name CHAR(30) PRIMARY KEY,
  address VARCHAR(255),
  presC INT,
  FOREIGN KEY (presC) REFERENCES MovieExec (cert)
);
```

Insert

```
INSERT INTO
  Movies
VALUES
  (
    "Screaming",
    Date '2023-10-31',
    "Adventure",
    "CSUC"
  );
```

AS

```
SELECT
  title AS name
FROM
  movies;

SELECT
  title AS name,
  length * 0.0167 AS lengthInHours;
```

LIKE

-% is regex *

- _ is regex .

NULL

- Arithmetic on NULL results in NULL
- Comparison on NULL results in UNKNOWN

Check if NULL

```
X IS NULL;
X IS NOT NULL;
```

ORDER BY

```
SELECT
  *
FROM
  Movies
WHERE
  YEAR = 1990
ORDER BY
  length,
  title;

SELECT
  *
FROM
  R
ORDER BY
  A + B DESC;
```

JOINS

```
SELECT
  MovieStar.name,
  MovieExec.name
FROM
```

```

    MovieStar,
    MovieExec
WHERE
    MovieStar.address = MovieExec.address;

```

Tuple Variables

```

SELECT
    Star1.name,
    Star2.name
FROM
    MovieStar Star1, --AS
    MovieStar Star2 -- AS
WHERE
    Star1.address = Star2.address
    AND Star1.name < Star2.name;

```

Union, Intersection, Difference

UNION, INTERSECT, EXCEPT

Duplicates in UNION, INTERSECT, EXCEPT

- SQL union, intersection, and except operations **eliminates** duplicates by default.
- To keep duplicates, use ALL

```

(
    SELECT
        title,
        YEAR
    FROM
        Movies
)
UNION ALL
(
    SELECT
        movieTitle AS title,
        movieYear AS YEAR
    FROM
        StarsIn
);
R INTERSECT ALL S
R EXCEPT ALL S

```

UNION

```

(
    SELECT
        title,
        YEAR
    FROM
        Movie
)
UNION
(
    SELECT
        movieTitle AS title,
        movieYear AS YEAR
    FROM

```

```
    StarsIn  
);
```

Intersect

```
(  
    SELECT  
        name,  
        address  
    FROM  
        MovieStar  
    WHERE  
        gender = "F"  
)  
INTERSECT  
(  
    SELECT  
        name,  
        address  
    FROM  
        MovieExec  
    WHERE  
        netWorth > 1000000  
);
```

Except

```
(  
    SELECT  
        name,  
        address  
    FROM  
        MovieStar  
)  
EXCEPT  
(  
    SELECT  
        name,  
        address  
    FROM  
        MovieExec  
);
```

Duplicates

```
SELECT DISTINCT  
    name  
FROM  
    MovieExec,  
    Movies,  
    StarsIn;
```

Aggregation

Operators

- SUM
- AVG
- MIN
- MAX

- COUNT

COUNT

COUNT(*) - counts tuples COUNT(DISTINCT x) - counts distinct in col

```
SELECT
  COUNT(*)
FROM
  StarsIn;

SELECT
  COUNT(DISTINCT starName)
FROM
  StarsIn;
```

LIMIT

```
SELECT
  title
FROM
  Movies
LIMIT
  5;
```

Grouping

```
SELECT
  studioName,
  SUM(length)
FROM
  Movies
GROUP BY
  studioName;
```

HAVING

Print total film length for only those producers who made at least one film prior to 1930

```
SELECT
  name,
  SUM(length)
FROM
  MovieExec,
  Movies
WHERE
  prodC = cert
GROUP BY
  name
HAVING
  MIN(YEAR) < 1930
```

Subqueries

```
SELECT
  name
FROM
  MovieExec
WHERE
  cert = (
    SELECT
      prodC
```

```
FROM
  Movies
WHERE
  title = "Star Wars"
);
```

Relation Conditions

- EXISTS
- IN
- ALL: true for all items
- ANY: true for any item
- NOT IN

Good Design

Faithfulness

Whatever relationship should make sense relative to real world being modeled

Avoiding Redundancy

Say everything once only

Simplicity Counts

Avoid introducing more elements into design than necessary

Choosing the Right Relationship

Picking the Right Kind of Element

Anomalies

Update Anomalies

Change info in one tuple but leave same info unchanged in another

Normal Forms

1NF

- Each row is unique (primary keys)
- Order of data does not imply anything
- Atomicity of data elements (no sets/lists)
- One datatype per column
- Column names are unique

2NF

- No partial dependencies
- Any attribute not part of primary key must depend on the primary key only
- Cannot depend on other keys, including subset of primary key
- Single candidate key

3NF

- No transitive FDs for non-prime attributes
- Non-prime attributes can only be derived by candidate key

BCNF

- All FDs for a relation must be the primary key or a super key of that primary key
- No reflexive dependencies
- Prime attributes can only be derived by candidate key

Decomposition

1. Elimination of Anomalies
2. Recoverability of Info: Can we recover original relation from tuples in its decomposition?
3. Preservation of Dependencies: if check projected FDs in decomp, could reconstruction joining satisfy FD

Lossless Join

Joining decomposed relations with natural join goes back to original

Dependency Preservation

All FDs from R can be expressed in at least one of the decomp of R

Minimal Basis

Aka minimal set

Set of FDs for a relation where there are no redundant FDs

Redundant:

$B \rightarrow A, BC \rightarrow A$